ASTRA Production

An Autonomous Architecture for Scientific Theorem Reasoning and Validation

Nelson Bolívar
Science-Stack Research Group

Architecture version: July 2026

Abstract

ASTRA Production (*Autonomous Symbolic Theorem Reasoning Architecture*) is a multi-agent research engine that turns scientific intuition into a falsifiable hypothesis, translates the hypothesis into an executable computational experiment, and critically analyzes the resulting evidence before accepting a conclusion. Its central design principle is the separation of language generation from evidence: language models propose, formalize, and analyze, while a symbolic or numerical computation oracle executes the validation program and returns reproducible evidence. The system supports isolated cycles and autonomous research campaigns, multiple model providers, local and remote mathematical engines, automatic code correction, human theorem approval, and persistent reports. This document describes the currently implemented architecture, its guarantees, and its limitations.

1 Motivation and epistemic principle

Language models are useful for formulating hypotheses, connecting domains, and writing programs, but a plausible response is not a proof. ASTRA therefore adopts an explicit separation of responsibilities:

- the **heuristic layer** proposes conjectures, generates code, and explains results;
- the **execution layer** computes with actual symbolic, numerical, or logical engines;
- the **control layer** preserves state, applies retries, guards verdicts, and requests human decisions;
- the **documentation layer** records code, output, analysis, providers, and research evolution.

ASTRA does not claim to automate mathematical truth. It turns AI-assisted reasoning into an inspectable process:

$$\begin{aligned}\text{intuition} &\longrightarrow \text{falsifiable hypothesis} \longrightarrow \text{program} \\ &\longrightarrow \text{executed evidence} \longrightarrow \text{reviewable verdict}.\end{aligned}$$

Model output never replaces execution. Conversely, successful execution is not sufficient by itself: the program must faithfully represent the claim and produce a signal that discriminates validation from refutation.

2 Current architecture

ASTRA runs as a local application with a browser interface. A Flask server exposes the interface and control API, while an asynchronous orchestrator maintains the state machine in the background. Agents communicate with LLM providers through a common client layer, and the oracle routes generated code to the appropriate computation engine. The browser interface, MCP server, and subprocess tool invoke the same canonical cycle; they do not maintain parallel scientific implementations.

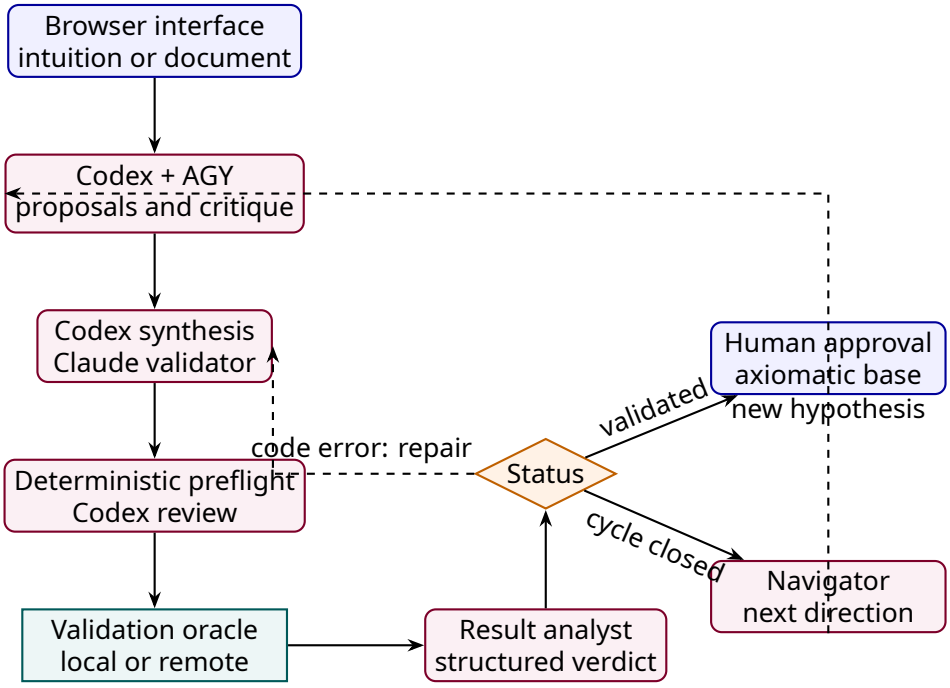

Figure 1: Logical flow of ASTRA Production. The Navigator participates in Research Loop mode, while human approval protects admission of results to the axiomatic base.

2.1 Input and interface

The user may enter a claim directly or upload PDF/TXT material. The application provides two operating modes:

– **Single Cycle**: evaluates one concrete intuition through a complete pass.

– **Research Loop**: decomposes a macro-level question into a sequence of related conjectures and preserves alternative branches.

The interface displays status, the current conjecture, generated script, oracle output, axiomatic base, reports, and event log. It also lets the user stop execution, approve or reject results, and configure providers for each role. The cycle cache includes the shared objective, axiomatic base, reflective trajectory summary, distance from the last milestone, and non-secret model/control configuration. A repeated direction in a changed scientific context therefore cannot replay a stale navigation decision.

2.2 Conjecture agent

Codex GPT-5.6 Sol and AGY receive the same final objective, current direction, and accumulated context. They propose in parallel, each adversarially critiques its rival proposal, and

Codex synthesizes one falsifiable conjecture with variables, assumptions, domain, and success criterion. In iterative mode they also use established theorems and previous refutations to avoid reopening closed paths.

2.3 Formal translator

The translator converts the conjecture into a self-contained validation script. The program must compute a discriminating quantity: a symbolic residual, identity, initial-condition check, inequality, counterexample, tensor property, or numerical comparison with explicit tolerances. It may declare the engine through an `ASTRA_ENGINE` header and suggest local or remote execution through `ASTRA_ORACLE`.

The translator does not issue the scientific verdict. It constructs the experiment by which the claim can be decided.

2.4 Deterministic preflight and independent review

Before execution, deterministic checks detect patterns that could produce a false validation, import defects, and known operational failures. Codex then compares Claude's script with both the shared objective and the exact conjecture. When a defect is locally repairable, Claude receives bounded instructions and returns an exact-replacement patch; Codex does not become the validator author. Reviewer rejection or an inapplicable patch yields an abstention or tool error, never scientific evidence.

2.5 Validation oracle

The oracle executes the program and captures return code, standard output, errors, and engine metadata. Python provides the common foundation, with libraries including SymPy, Z3, NumPy, SciPy, mpmath, EinsteinPy, Pint, fluids, and QuTiP. ASTRA can also route jobs to SageMath, Maxima, or Cadabra when available. On Windows, `ASTRA_WSL_DISTRO` pins the intended WSL2 distribution so that execution does not silently move when the machine's default distribution changes. The production workstation uses Debian/WSL2 for SageMath, Maxima, Cadabra, and the local Lean/Mathlib environment.

Execution may be:

– **local**, suitable for symbolic algebra and moderate computation;

– **remote**, through an SSH-connected Linux worker for heavy workloads, GPU use, or tools installed on the compute workstation.

The policy may be fixed by configuration or inferred from markers and script characteristics. A missing optional engine must cause an explicit failure rather than a fabricated validation.

2.6 Analyst and verdict guard

The analyst receives the conjecture and raw evidence. It returns a structured response with one of four primary states:

State	Meaning
`VALIDATED`	Computational evidence supports the stated hypothesis.
`REFUTED`	Execution finds a counterexample or contradicts the hypothesis.
`CODE_ERROR`	The program failed to execute the intended test correctly.
`API_ERROR`	The LLM provider, network, or relevant quota failed.

When a CODE_ERROR occurs, ASTRA requests corrected code and reruns validation up to the configured limit. The verdict guard restricts conclusions that are incompatible with execution evidence. This reduces, but does not eliminate, the risk that the analyst will turn a technical failure into a scientific claim.

2.7 Human approval and axiomatic base

A hypothesis marked as validated remains pending. Only after researcher approval is it added as an established theorem. Refutations and their reasons also enter the working context so that discarded paths are not silently repeated. This human gate recognizes that program correctness, physical assumptions, and the scope of a conclusion still require expert judgment.

3 Autonomous research loop

During a campaign, ASTRA maintains a *Research Session* associated with a macro question. After each cycle, the Navigator assesses progress and proposes:

– the next depth-first research direction;

– parallel branches worth preserving;

– whether a milestone requiring review has been reached;

– whether the macro question may be considered resolved.

If a conjecture is validated, the system may generalize it, explore its boundaries, or apply it to another regime. If it is refuted, the system seeks an alternative that avoids the specific cause of failure. The researcher may accept the proposal, redirect the investigation, or activate a saved branch. In autonomous mode ASTRA passes milestones without pausing until the Navigator declares resolution, the maximum runtime is reached, or the user stops the session.

This is a depth-first procedure with branch memory; it is neither an exhaustive search nor a guarantee that a resolution declaration is correct.

4 Controlling ASTRA from an agent

ASTRA is not limited to its browser interface. Its Model Context Protocol (MCP) server exposes the oracle as tools callable by Codex, Claude Code, Gemini, or another compatible client. The agent conversing with the researcher can therefore plan the test, write the validation program, request execution, and inspect evidence without manually moving results between applications.

The current interface provides six principal operations:

Tool	Function
`astra_execute`	Runs an existing script on the local, remote, or automatic oracle.
`astra_cycle`	Runs conjecture, translation, oracle, and analysis as one complete cycle.
`astra_probe`	Reads the phase and heartbeat of a cycle without interrupting it.
`astra_submit`	Submits long computation as a detached job.
`astra_job`	Lists or polls jobs, incremental output, and final results.
`astra_status`	Checks availability of the remote worker.

The MCP server is registered by pointing the client to ASTRA's environment interpreter and `mcp_server/server.py`; a new agent session is then opened so that the tools are loaded. The researcher may ask, for example, "construct an independent test of the Ricci scalar, run it with `astra_execute`, and do not accept the result until the residuals have been inspected." Computations longer than a few minutes should use `astra_submit` and be followed with `astra_job`.

This mode makes the conversational agent the planning layer and ASTRA the traceable execution layer. The agent receives no additional authority: it must still expose code, assumptions, output, and the decision criterion.

5 Extension through scientific engines and packages

The oracle layer is extensible rather than a closed list. Any package available in the execution environment may participate in validation. In-house scientific packages include:

`GR_python` `pyWarpFactory` `warp_bubble_optimization`.

Integration has a simple contract: the script must import the package, fix reproducible inputs, produce inspectable evidence, and end with an explicit verdict. The same package must be installed in the selected local or remote environment.

This mechanism allows ASTRA to reuse validated solvers, metrics, optimizers, energy-condition analyses, and test suites. Important conclusions should combine independent legs whenever possible—for example, a symbolic identity, fixed-seed numerical evaluations, and a known limiting case.

5.1 Lean 4 and Mathlib

Lean 4 is integrated as an optional engine through the marker `# ASTRA_ENGINE: lean`. The translator may generate a file importing Mathlib; ASTRA passes it either to a pinned native project, to the pinned Debian/WSL2 project, or to the configured ASTRUM formal route. The production local route fixes Lean 4.30.0 and the Mathlib commit beginning `c5ea00351c28`. Acceptance by the Lean kernel provides mechanical verification suitable for formal logic, discrete mathematics, algebra, and theorems that can be represented in the proof assistant's language.

Lean does not replace numerical physics calculations or prove that a formalization faithfully represents a physical system. Its guarantee applies to the formal statement and imported axioms. If the compiler rejects a proof, ASTRA classifies the event as a code error and may request a correction; rejection must never be turned into a refutation of the theorem.

5.2 Wolfram Mathematica as a complementary oracle

When the model has access to a Wolfram plugin or the `mathematica-agent-bridge` project, the agent can write and execute Wolfram Language directly. The local bridge connects external processes to a Mathematica kernel through a JSON queue and can operate on an open notebook or headlessly through `wolframscript`. Its actions include `evaluate-wl`, `simplify-symbolic`, `compare-expressions`, `commutator`, and `gr-curvature`, as well as reading, writing, and evaluating notebook cells.

Mathematica can act as a second independent validator: the agent runs a test through ASTRA and repeats the identity, tensor, or limit through the bridge. Agreement between independently implemented engines strengthens the evidence; disagreement triggers inspection of conventions, domains, and assumptions. The bridge evaluates code with the local

user's privileges and should therefore be enabled only for trusted agents and command folders.

6 Providers, engines, and deployment

Roles may be reassigned for controlled ablations, but the compact production contract is explicit: Codex CLI with `gpt-5.6-sol` at `xhigh` effort proposes, synthesizes, reviews, and analyzes; AGY with `gemini-3.1-pro-high` at `high` effort co-proposes, critiques, and navigates; Claude Code with `claude-opus-4-8` writes and repairs code. The `scripts/audit_architecture.py` command verifies this map, the binaries, epistemic gates, local scientific engines, and configured formal routes without reading or printing secrets. `ASTRA_REQUIRED_LOCAL_ENGINES` makes missing production engines a blocking audit failure. The idempotent `scripts/bootstrap_wsl_scientific_stack.ps1` script recreates the pinned Debian stack. Other supported providers are reserved for controlled comparisons.

The desktop installation creates a Python environment and launches the interface at `http://127.0.0.1:5050`. Credentials and oracle parameters remain in local configuration and must not be committed to the repository. ASTRA stores sessions and reports in the application's workspace.

7 Persistence, traceability, and calibration

Each cycle produces HTML and Markdown reports containing the intuition, conjecture, script, execution result, analysis, and selected providers. Investigations can be saved, reloaded, and reviewed through history. The benchmark suite covers general relativity, ordinary differential equations, fluid mechanics, logic, quantum systems, and symbolic computation.

Benchmarks serve two purposes: they verify that the engines remain operational and compare how well different providers construct discriminating tests. They do not replace scientific validation of a new problem, but they expose infrastructure regressions and systematic biases.

8 Guarantees and limitations

ASTRA improves reproducibility because it preserves and executes the program, but it does not automatically turn a computational check into a universal proof. Several obligations remain:

– verify that the formal conjecture matches the original question;

– inspect whether the code tests a weakened or circular version of the claim;

– distinguish numerical evidence, symbolic identity, and logical proof;

– state domains, units, boundary conditions, and tolerances;

– repeat results that depend on precision, grid, or platform;

– subject consequential conclusions to independent human review.

Thus, `VALIDATED` means "validated by the oracle under the recorded program and assumptions," not "absolute mathematical truth." ASTRA's strength lies in making the evidence chain visible and repeatable.

9 Conclusion

ASTRA Production has evolved from a fixed processing scheme into a configurable, iterative, multi-agent research architecture. Formulation, translation, execution, analysis, navigation, and approval are distinct responsibilities. The oracle provides a reference external to model discourse; reports preserve traceability; and the human gate prevents autonomy from being mistaken for scientific authority. The result is a platform for conducting research with language models without silently delegating to them the final criterion of truth.